Alex Keys (keysa)
Johnny Zheng (jnzheng)
Mitchell Kugelberg (mikkugel)
Jack Wember (jwember)

Texas Hold ‘em Bot

Our AI is designed for someone to play no limit Texas Hold ‘em against it. The AI follows all the rules of Texas Hold ‘em, while also trying to find the best possible move during each available turn.

Problem Space

The rules of Texas Hold ‘em are fairly simple to understand once you get a grasp of it. The win condition of Hold ‘em is to either force your opponent(s) to fold or to have the best 5-card combination at showdown (the last turn of the match). Before each game officially begins, “blinds” are posted (a required minimum bet for each player to put up before a match). There is a “little blind” and a “big blind”, where the little blind sets up the initial bet and the big blind has to double that bet. After that, each player receives 2 “hole” cards that only they can see. Then, community cards are dealt with the order being: 3 initial cards (the flop), the 4th card (the turn), and the 5th and final card (the river). Between each dealing of cards from the dealer, any player in table order can decide to call (match the current highest bet), raise (match the current highest bet + add an additional bet), or fold (surrender their bet). The game does not end until every player but 1 folds (or in our case just 1 player), or until showdown where each player hopes for the best 5-card combination in order to win.

There were some constraints that were run into during development. Firstly, having a full No-Limit Heads Up Texas Hold ‘em environment can get pretty big to manage (many bet sizes, card combinations), and trying to solve that with full CFR would be incredibly difficult. Another constraint was time. Juggling other projects, several exams, events, all while trying to build something worthwhile proved to be difficult.

Our main objectives were to have a working heads up no limit Texas Hold ‘em environment with at least two distinct algorithms/techniques and be able to collect performance metrics and compare them. Some major challenges included imperfect information (as with the nature of poker/hold ‘em, you are not allowed to see other players’ cards), algorithm research/understanding and implementation, and again, time management.

Variations that could be encountered are: different opponent styles (being aggressive/passive/unpredictable), and time vs. accuracy in algorithms (balancing how long the program takes to run vs. how accurate the decision making is).

Techniques Implemented

There are different techniques/algorithms implemented in this project.

Monte Carlo Equity Simulation: One implementation the bot uses is Monte Carlo Simulations. It runs a set number of trials and evaluates hand equity (encapsulates “how many times did this hand win/tie/lose?” into a usable float) vs a random opponent and uses pot-odds logic decision making. When facing an opponent bet it evaluates using the equation: (to_call/(pot + to_call)) (this is the classic (risk/(reward + risk)) equation). When not facing a bet, the bot will either bet or raise within a certain threshold.

The time complexity is O(n), where n is the number of trials the user would like to run. Space complexity would be O(1).

Limitations of the Monte Carlo implementation could be that it's a pretty “surface” level decision making system and not incredibly complex. It’s not exactly a “strategy” but goes off of general hand strength. The implementation will also become much slower if you wish to increase accuracy. It will work, but maybe not the best.

Expectiminimax: The expectiminimax component is intentionally minimal. It evaluates a compact action set (fold/call/check/raise depending on state) using a one layer expected value model. Hidden chance is done through Monte Carlo equity sampling instead of full chance-tree expansion. Having this gives a useful baseline for comparison. It is algorithmically distinct from pure pot-odds thresholding but remains simple enough to run quickly and inspect.

The time complexity is O(b^m), where b is the branching factor and m is the max depth of the game tree. Space complexity would be O(b * m) with b and m being the same as before.

Limitations of expectiminimax in our implementation would be that it has simplified EV assumptions, and only looks one move ahead at a time.

External Sampling Monte Carlo Counter Factual Regret Minimization: Uses regret matching to get a mixed strategy at each information set. Rather than enumerating the full tree, it samples opponent actions and limits full branching which helps with manageability. CFR maintains regret values at each unique info set (which are game states) for each action. During training, it repeatedly samples hands and decision trajectories, updating regrets based on the difference between the outcome of each action they take.

Per iteration, the time complexity is O(|A|max * D * X) with |A|max being the max number of actions at any infoset, D being the depth of the game tree, and X being the number of externally sampled actions (like dealing or opponent actions). Space complexity is O(I * |A|) with I being the number of infosets, and |A| being the number of actions available at that infoset.

Limitations of MCCFR: it needs lots and lots of training to be effective, and is incredibly complex to implement (with too few iterations, the strategy gets noisy and can seem random).

Alternatives Considered:
Full CFR (too big NLHE purposes, and incredibly time intensive)

How our solution models the Human Thought Process

When humans play poker they typically run through each hand considering the most likely possibilities given the cards at play. Weighing those possibilities, they make what they believe is the best choice with the information available. Our bot mirrors this thought process in the way it reasons logically about the game.

The Monte Carlo simulation models the way a player thinks through their opponent's potential hands. It does this better than a human because it can simulate thousands of possible outcomes rather than just the most likely ones. By averaging those results, the bot gains much clearer insight into the right decision than a human could reach on their own.

The breakeven equity calculation reflects the pot odds reasoning that experienced players do naturally at the table. When a player looks at the pot and the amount they need to call, they are asking whether it is worth continuing. Our bot answers that same question mathematically. The raise threshold, requiring equity above 0.70 before betting, captures the human tendency to play aggressively only when confident in hand strength. It also does something humans like to do, which is bluff. No matter the strength of its hand, the bot will bluff a little bit of the time and mess with its opponent. Finally, re-evaluating equity after each new community card mirrors how a human continuously updates their read on a hand as new information is revealed.

Expectiminimax mirrors the way players mentally project a hand forward, accounting for both opponent decisions and the randomness of cards that have yet to be dealt. Just as a human might think "if I bet here, my opponent might call or fold, and then any card could fall on the river," expectiminimax models this branching structure by alternating between the player's choices, the opponent's responses, and chance events like community card draws.

The CFR bot resembles learning frequencies. Humans often learn that some actions should be taken just “some of the time” to avoid being predictable. CFR will explicitly learn probabilistic action frequencies at different situations and be able to play in a mix of styles.

Empirical Analysis

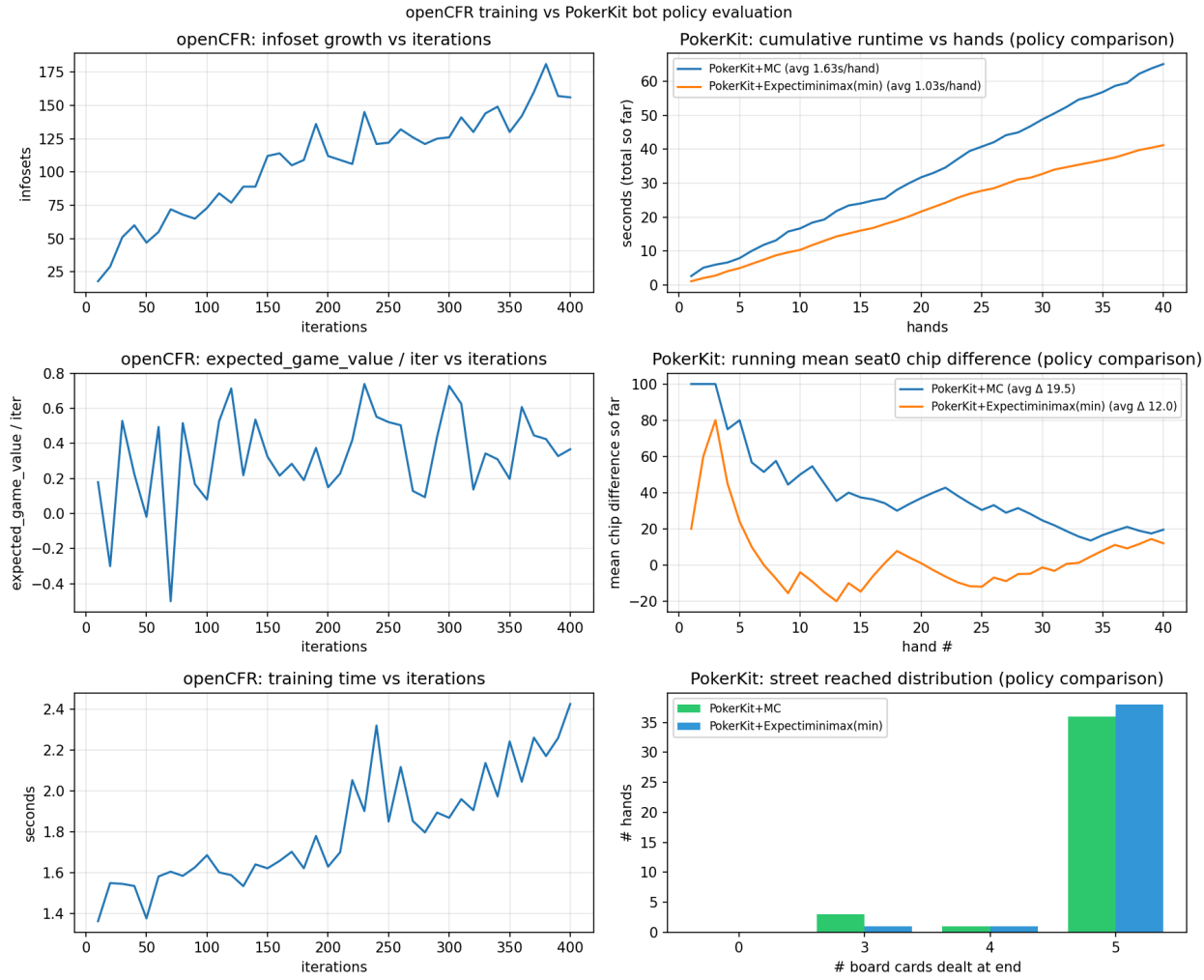
The Monte Carlo bot performs well here because it directly estimates equity and uses pot-odds logic when facing bets, which is especially effective against a simple call-heavy baseline where straightforward value betting can be profitable.

The expectiminimax-min policy is quicker because it uses a shallow one-layer EV comparison with a smaller action model, but that also limits the depth of its strategy and can reduce chip performance in this environment.

These results are consistent with expectations. Simulation-heavy decision-making (like monte carlo equity with repeated rollouts) usually gives stronger decisions but costs more runtime per hand. A lightweight policy usually runs faster, but performance depends heavily on the quality of abstraction and how well it matches opponent behavior.

The openCFR graphs also reinforce the above. Infosets grow with iterations, training time increases, and expected value per iteration is volatile at smaller training scales, meaning that getting a clearer picture of how the algorithm actually does would require much more training and/or better abstraction design.

Defining and evaluating no-limit heads-up hold’em is tough because performance is very sensitive to bet sizing choices, opponent model assumptions, and abstraction structure. Small implementation changes can heavily impact results, and short experiments are unreliable and noisy due to high variance. It is relatively easy to build something that is fast or something that is strong in a narrow benchmark, but getting both speed and decision strength together requires better abstractions, stronger opponent modeling, and larger controlled evaluation runs.


openCFR training vs PokerKit bot policy evaluation
openCFR: infoset growth vs iterations
infosets
iterations
PokerKit: cumulative runtime vs hands (policy comparison)
PokerKit+MC (avg 1.63s/hand)
PokerKit+Expectiminimax(min) (avg 1.03s/hand)
seconds (total so far)
hands
openCFR: expected_game_value / iter vs iterations
expected_game_value / iter
iterations
PokerKit: running mean seat0 chip difference (policy comparison)
PokerKit+MC (avg Δ 19.5)
PokerKit+Expectiminimax(min) (avg Δ 12.0)
mean chip difference so far
hand #
openCFR: training time vs iterations
seconds
iterations
PokerKit: street reached distribution (policy comparison)
PokerKit+MC
PokerKit+Expectiminimax(min)
hands
board cards dealt at end

Outside Code

No code was directly copied from StackExchange or StackOverflow or elsewhere. We did not paste external algorithm implementations line-for-line into our files. Most of the project logic (Monte Carlo equity decision policy, expectiminimax policy, game loop integration, analysis flow, and terminal menu) was completely hand written and learned through tutorials and reading documentation.

The project does rely on third-party libraries as dependencies or for analysis, and those are used as infrastructure rather than copied snippets. PokerKit is used for legal game state management, card/deck handling, and hand evaluation which lets our code focus on decision logic instead of rebuilding poker rules from complete scratch. openCFR is used for the External Sampling MCCFR demo/training component and metrics analysis. NumPy is used for efficient arrays, and matplotlib is used for plot generation in comparison_outputs/distributions.png. Our code calls library APIs and uses their outputs, but we did not steal from someone's work.

PokerKit. Universal, Open, Free, and Transparent Computer Poker Research Group: https://pokerkit.readthedocs.io/en/stable/index.html
openCFR. Rex Stockham: https://github.com/stockhamrexa/openCFR
NumPy. Travis Oliphant: https://numpy.org/
Matplotlib. John D. Hunter: https://matplotlib.org/

Expansions/Improvements

The clearest thing to do next is to strengthen decision making while keeping runtime as low as possible. For the Monte Carlo policy, we would improve opponent modeling beyond uniform hidden-card assumptions, add better action abstraction (more realistic raise sizing choices), and have multiple opponent styles instead of one passive baseline. For the expectiminimax-min policy, the biggest upgrade would be deeper lookahead than one layer with selective pruning so we can preserve speed while improving action quality on later streets. In terms of external sampling MCCFR, having the openCFR implementation work within the PokerKit environment would allow for fair testing and comparison between all three algorithms.

The main lessons that were learned here are about engineering tradeoffs. First, using a reliable state manager (PokerKit) is critical. Legal action correctness can easily break if game rules are hand implemented under time pressure. Second, abstraction design heavily controls both speed and quality, so "fast vs strong" is usually a modeling decision, not just a coding decision. Third, thorough testing and analysis is a requirement. Without having repeated iterations and hands, it can be easy to misinterpret data that is simply noisy.